P3A Research report :
Training the first layer is all you need

Hugo VIDAL
supervised by Florent Krzakala
IdePHICS Lab

April - July 2025

Abstract — In this report, we investigate the surprising capability of fully connected neural networks to achieve strong performance, even when only the first layer is trained. Through experiments on synthetic teacher-student settings and real image datasets, we show that partial training of the network, possibly combined with last-layer fine-tuning, can match or even surpass fully trained models. These findings challenge conventional training paradigms and may open the way to new directions for efficient learning.

Table of contents

Introduction

Effectively managing noisy or incomplete data represents a significant and ongoing challenge in both everyday life and various technological applications. Real-world data rarely come in perfect form, for instance, weather forecasts rely on incomplete or noisy sensor data, online shopping recommendations depend on uncertain customer preferences, medical diagnostics often involve interpreting unclear, heterogeneous or incomplete patient data. The ability to reliably extract meaningful insights from imperfect data is essential in many disciplines from daily activities to scientific ones.

The theory of inference involves extracting meaningful information from data that may be noisy, incomplete, or high-dimensional. It encompasses a wide range of problem types, including :

- **Statistical detection and estimation :** Inferring global properties of a population using only partial observations, such as estimating average behaviors or trends from a limited sample.
- **Graph inference and network reconstruction :** Uncovering the structure of complex networks, such as identifying communities in social or biological systems, or mapping interactions between elements in a system.
- **Compressed sensing and sparse recovery :** Recovering essential information from incomplete or undersampled data, for example in image compression or real-time signal processing.
- **Clustering and community detection :** Grouping data points into subsets that share similar features or behaviors, such as detecting functional loci in gene networks.
- **Regression and classification :** Predicting outcomes based on input features without necessarily knowing the exact relationship between them, for example forecasting weather, simulating physical fields, or classifying images.

For each of these problem types, physicists, mathematicians, and computer scientists develop tools and algorithms (ex : estimators, Louvain method, PCA, K-means, neural networks...) designed to extract accurate insights from data. They also investigate how these algorithms perform relative to specific parameters (such as the dimension of data, the amount of noise...) and conditions of the problem.

---

[1]Note that generation, in a broad sense, can be interpreted as a form of regression—modeling human behavior in speech and reasoning, or mapping disordered inputs to structured outputs in image generation.

Several key concepts play a central role in these investigations. The **statistical threshold** defines conditions under which it becomes theoretically possible to reliably extract information from noisy or incomplete data. **Computational complexity** describes how the resources required scale with the increasing size or difficulty of a problem. The **sample complexity** specifies the minimum quantity of data needed to achieve inference. Additionally, **uncertainty characterization** assesses the confidence or reliability of outcomes produced by methods and algorithms. Together, these concepts help clarify the practical and theoretical limitations regarding feasibility, efficiency and confidence of inference tools.

Neural networks are powerful tools for inference tasks because of their ability to capture complex, nonlinear relationships in high-dimensional data. They have first emerged as a classification and regression tool. But two major breakthroughs have significantly expanded the scope of neural networks to generation tasks[1]. First, Generative Adversarial Networks (GANs) (Goodfellow et al., 2014) introduced an adversarial training framework that enables the generation of highly realistic data, inspiring extensive research in generative modeling. Secondly, Transformers (Vaswani et al., 2017), which revolutionized sequence modeling and now form the backbone of many neural networks applications. These innovations have pushed deep learning beyond basic predictive tasks, allowing for high-quality generative capabilities and the emergence of reasoning-like behavior in LLMs.

With the rapid and widespread adoption of neural networks in increasingly impactful applications, the need to better understand their inner workings has become essential.

However, neural networks remain complex and often counterintuitive systems. For a long time, they were considered theoretically problematic due to several challenges :

1. The large number of parameters was expected to lead to overfitting of the training dataset, seemingly preventing good generalization on unseen datas.
2. The optimization algorithms aims to minimize a non-convex loss function, which theoretically should not be efficient with computationally reasonable algorithms.
3. Their complexity makes it extremely difficult to predict fundamental properties such as statistical thresholds, sample complexity, or uncertainty of the output in realistic settings.

Nevertheless, in practice, neural networks achieve surprisingly strong generalization and performance, with a computationally low cost algorithm (stochastic gradient descent), defying earlier expectations. This paradox has driven an intense research effort to understand their behavior and to develop theoretical tools that can predict or explain key properties.

Research in this area generally follows two fundamentally different approaches. Academic research tends to start from simplified models, investigating how learning emerges in controlled, idealized settings—similar to how physicists analyze simplified systems to gain fundamental insights before adding complexity. This approach seeks to identify the learning regimes, generalization properties, and dynamics of training. In contrast, industrial research often focuses on large-scale models. It combines engineering heuristics with empirical exploration, frequently discovering new capabilities through experimentation and large-scale training.

During my internship, I was part of the IdePHICS Lab, which follows the academic approach. My work focused on shallow neural networks, specifically classical fully connected architectures. This research was motivated by recent observations from the lab (Dandi et al., 2025) (**see I.**), where it was noted that neural networks can still learn even when only part of the architecture is updated during training. These observations led to a fundamental question :

Can a fully connected neural network learn efficiently even if only its first layer is trained ?

My project aimed to explore this question by designing and analyzing training regimes in which only the first — and eventually the last — layer of the network is updated, while keeping the rest of the architecture fixed. The goal was to assess whether the initial observations held consistently, and to determine to what extent such partial training can still result in meaningful learning and generalization.

1 Why focusing on training only the first layer ?

1.1 Setting the framework

To properly understand the motivation behind this study, let us begin by reviewing the key notions and notations that will be useful for good understanding of the problem.

1.1.1 General model for neural networks :

A neural network can be described as a parameterized function f_θ, where the set of parameters $\theta = \{\theta_1, \dots, \theta_p\}$ defines the network's behavior. Initially, these parameters are randomly initialized. To make the network perform a specific task, a learning algorithm adjusts θ based on a given set of data.

In supervised learning, the algorithm needs :

- A **dataset** $\mathcal{D} = \{(\mathbf{x}_\mu, \mathbf{y}_\mu)\}_{\mu=1}^N$, where $\mathbf{x}_\mu \in \mathbb{R}^d$ are the inputs and $\mathbf{y}_\mu$ the targets. It corresponds to a set of examples where the task has already been achieved.
- A **loss function** $\ell(f_\theta(\mathbf{x}), \mathbf{y})$ that quantifies the difference between prediction and ground truth.

The algorithm minimizes the empirical risk :

$$\mathcal{L}(f_\theta, \mathcal{D}) = \frac{1}{N} \sum_{\mu=1}^{N} \ell(f_\theta(\mathbf{x}_\mu), \mathbf{y}_\mu)$$

Most commonly, the optimization is performed via **stochastic gradient descent (SGD)**, which has surprisingly good capacity for non-convex optimisation for a low computational cost :

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta \mathcal{L}(f_\theta, \mathcal{D})$$

where η is the learning rate. Provided that the number of data N and iterations t are large enough, the algorithm reaches a parameter set θ^* for which the resulting function f_{θ^*} generalizes well — meaning it performs accurately on new, unseen inputs from the same distribution as the examples.

1.1.2 Fully Connected Neural Networks (FCNNs) :

A fully connected neural network (FCNN) is the most basic type of neural network in which f_θ is a sequence of affine transformations followed by nonlinear activations :

$$f_\theta(\mathbf{x}) = \mathbf{b}_L + W_L\,\sigma\left(\mathbf{b}_{L-1} + W_{L-1}\,\sigma(\dots\sigma(\mathbf{b}_1 + W_1\mathbf{x}))\right)$$

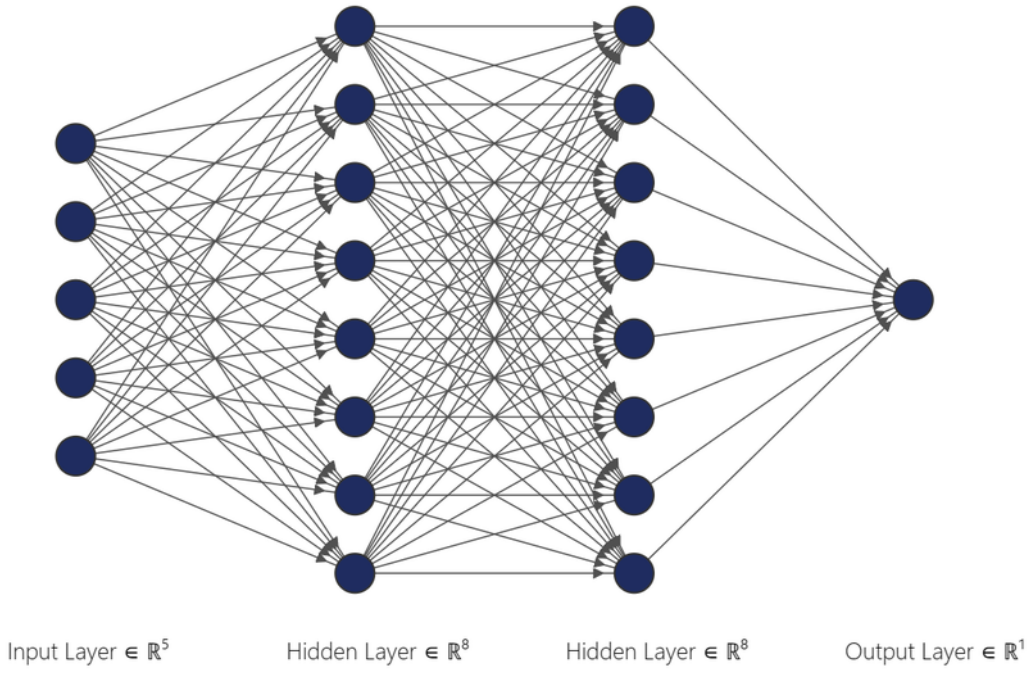


FIGURE 1 – Schematic representation of a FCNN

Each layer ℓ is defined by weight matrices W_ℓ and biases $\mathbf{b}_\ell$, and σ is a non-linear function (ex : ReLU). In classification tasks, a final activation like $\tanh$ or sigmoid is used.

Interpretation : For binary classification, the first $L-1$ layers transform the data to make it linearly separable and the final layer performs a linear classification in the transformed space. (see Fig.2)

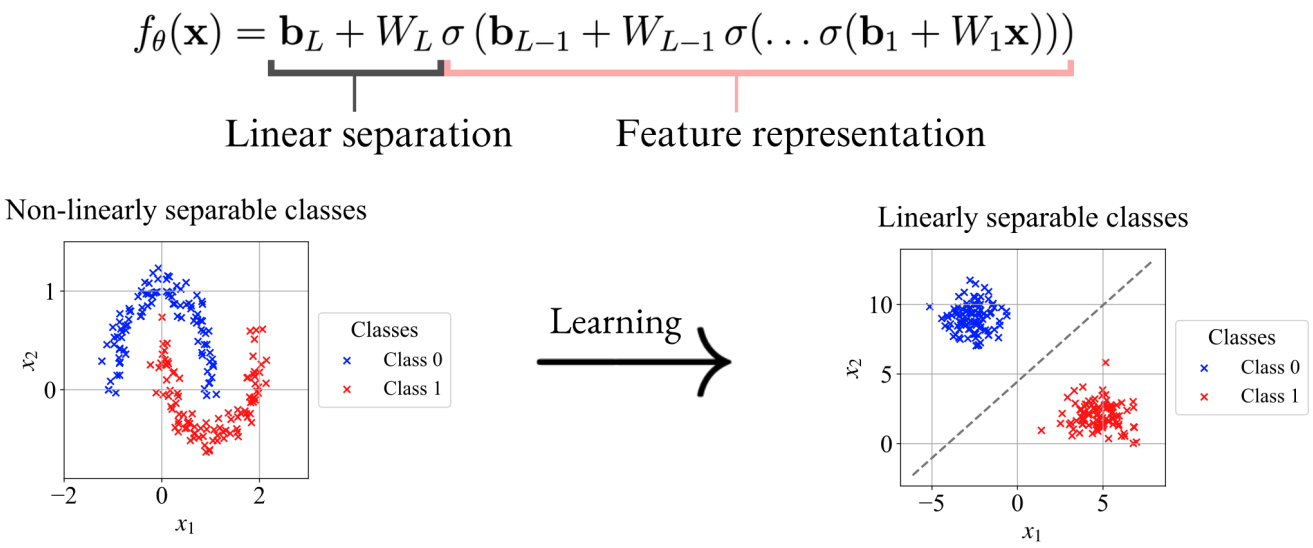


FIGURE 2 – Schematic functionning of a FCNN on a binary classification task

1.2 Motivation of the study

1.2.1 Prior work :

In the paper *Computational Advantage of Depth* (Dandi et al., 2025), the authors explore how depth enhances a neural network's ability to learn certain complex target functions. This is done using the **teacher–student framework**, in which a student neural network is trained to reproduce the behavior of a known function, the teacher.

In this kind of setting, the dataset is constructed as :

$$\mathcal{D} = \{\mathbf{x}_\mu, f^*(\mathbf{x}_\mu)\}_{\mu=1}^N, \quad \text{with} \quad \mathbf{x}_\mu \sim \mathcal{N}(0, I)$$

with the teacher function f^* used in the article is defined as :

$$\begin{cases} f^*(\mathbf{x}) = \varphi\left(\mathbf{a}_{L'-1}^\top P_{k,L-1}(\mathbf{h}_{L'-2}(\mathbf{x}))\right) \\ \mathbf{h}_\ell(\mathbf{x}) = A_\ell P_{k,\ell}(\mathbf{h}_{\ell-1}(\mathbf{x})), \quad \forall \ell \in [\![2, L'-2]\!] \\ \mathbf{h}_1(\mathbf{x}) = W^*\mathbf{x} \end{cases}$$

This function composes linear and nonlinear transformations in depth, creating a hierarchical structure. The authors show that :

- When the student has depth $L = L'$, it can learn the teacher with a number of data growing **polynomially** with the input dimension d;
- When the student has depth $L < L'$, the number of samples required becomes **super-polynomial**.

They explain this phenomenon by what they call **hierarchical learning**, which means that the ℓ-th layer of the student learns the ℓ-th layer of the teacher.

$$f_\theta(\mathbf{x}) = \mathbf{w}_3^\top \sigma(\mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x}))$$

$$f^\star(\mathbf{x}) = \tanh\left(\frac{\mathbf{a}^{\star\top} P_3\,(W^\star \mathbf{x})}{\sqrt{d^{\varepsilon_1 = 1/2}}}\right)$$

FIGURE 3 – Illustration of the hierarchical learning process. Each layer of the student network learns a part of the teacher function.

1.2.2 A surprising observation :

To illustrate this, the authors train a student with architecture :

$$\hat{f}_\theta(\mathbf{x}) = \mathbf{w}_3^\top \sigma\left(\mathbf{W}_2\, \sigma(\mathbf{W}_1 \mathbf{x})\right)$$

to learn the teacher :

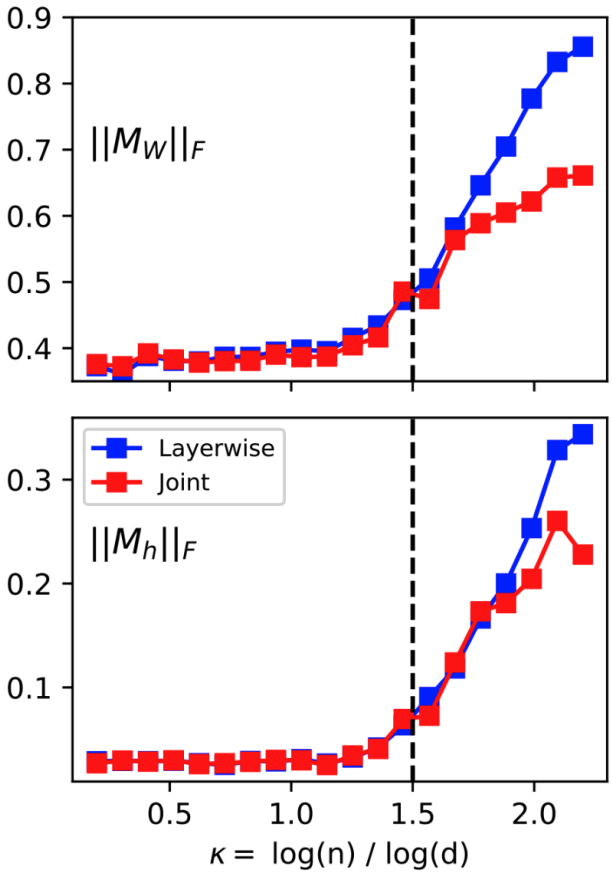
$$f^\star(\mathbf{x}) = \tanh\left(\frac{\mathbf{a}^{*\top} P_3(\mathbf{W}^* \mathbf{x})}{\sqrt{d^{\epsilon_1 = 1/2}}}\right)$$

First, they train the network by updating only the **first layer** W_1, leaving W_2 and $\mathbf{w}_3$ fixed and random. They measure two metrics are during the training :

- The correlation between W_1 and the first transformation W^* of the teacher.
- The correlation between second-layer activations[1] $h(\mathbf{x})$ and the corresponding component $h^*(\mathbf{x})$ in the teacher :

$$\begin{cases} \mathbf{h}(\mathbf{x}) = \mathbf{W}_2\, \sigma(\mathbf{W}_1 \mathbf{x}) \\ \\ \mathbf{h}^*(\mathbf{x}) = \mathbf{a}^\top P_3(\mathbf{W}^* \mathbf{x}) \end{cases}$$

[1] **Activation :** representation of the input at a given layer of a neural network.



$$M_W = \frac{W_1 W^{*\top}}{\|W_1\|_F \, \|W^*\|_F}$$

$$M_h = \frac{\mathbb{E}\left[\mathbf{h}(\mathbf{x})\,\mathbf{h}^*(\mathbf{x})\right]}{\sqrt{\mathbb{E}\left[\mathbf{h}(\mathbf{x})^2\right]\mathbb{E}\left[\mathbf{h}^*(\mathbf{x})^2\right]}}$$

FIGURE 4 – Correlation between the student and the teacher at layer 1 and layer 2 when training only the first layer W_1. n corresponds to the size of the dataset used for training and d corresponds to the dimensions of the inputs.

The first result (top graph) aligns with expectation : training the first layer improves the correlation with the first part of the teacher function. Surprisingly, however, on the bottom graph, one can see that the second-layer correlation also increases, despite W_2 is untrained.

This suggests that the first layer is optimized not only to perform its own local transformation, but also to shape its output in a way that facilitates alignment with the fixed, untrained upper layers. In other words,W_1 adapts during training to produce representations that are compatible with the following random layers, enabling them to approximate their respective targets in the teacher network.

Those observations lead to two fundamental questions :

Can a FCNN learn meaningful feature[1] representation even if only the first layer is trained ?

[1]**Feature :** property or characteristic of the input data that is used by a model to make predictions or decisions..

Since the article shows that a neural network benefits from depth to learn hierarchical teacher function, the second question is :

Can a FCNN benefit from depth even if only its first layer is trained ?

2 Experimental results on synthetic and real data

In order to address the two aforementioned questions, I conducted a series of experiments. This section presents the main outcomes obtained on both synthetic and real-world datasets.

2.1 Synthetic Data

We first analyze the behavior of various training strategies on a dataset designed with a known hierarchical structure between inputs and targets.

2.1.1 Teacher-Student Setup :

Teacher Function We use the same teacher function as proposed in the paper (Dandi and al., 2025) :

$$\begin{cases} \mathcal{D} = \{\mathbf{x}_\mu, f^*(\mathbf{x}_\mu)\}, \quad \text{with} \quad \mathbf{x}_\mu \sim \mathcal{N}(0, I) \\ f^*(\mathbf{x}) = \tanh\left(\dfrac{\mathbf{a}^{*\top} P_3(\mathbf{W}^*\mathbf{x})}{\sqrt{d^{\epsilon_1=1/2}}}\right) \end{cases}$$

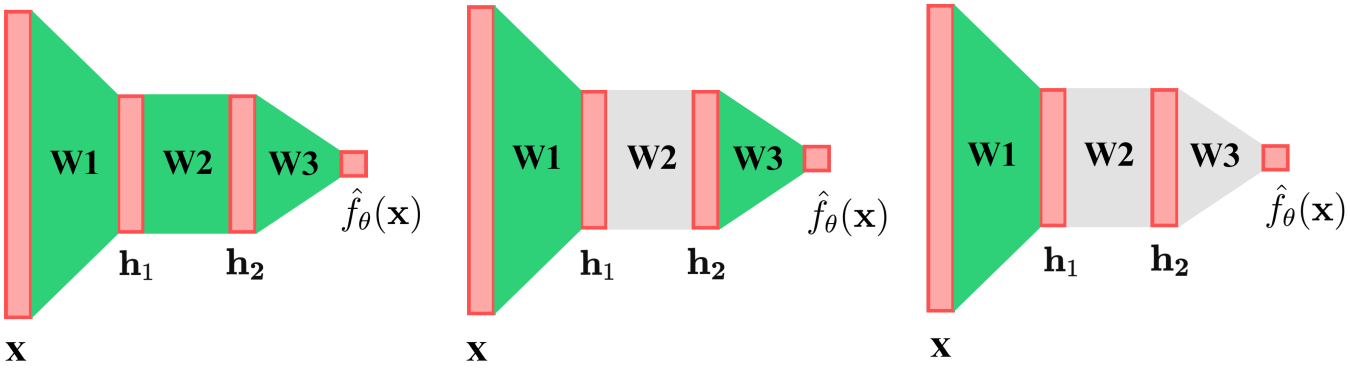
Student Networks We define several student models with varying depths $L \in \{2, 3, 4\}$ and fixed width $W = 512$ neurons per hidden layer. This allows us to investigate the influence of depth on learning performance under different training regimes.

2.1.2 Training Strategies :

We compare four different training procedures :

- **Full Training :** All layers of the student network are updated during training. This serves as a baseline for each depth level.

- **Extremities Training :** Only the first and last layers are trained, corresponding respectively to the feature extractor's entry point and the final linear separation layer (see Fig.2).
- **First Layer Only Training :** Only the first layer is updated during training. The rest of the network remains frozen. The goal is to adapt the initial feature extraction to match the fixed final layers.



(a) Full training (b) Extremities training (c) First layer only

FIGURE 5 – Schematic representation of the different training strategies.

- **First Layer + Last Layer Fine-Tuning :** This hybrid method follows four stages :
  1. Train only the first layer for N iterations.
  2. Freeze the first layer, copy the model, and fine-tune only the last layer for N' iterations.
  3. Evaluate and save the fine-tuned model.
  4. Discard the fine-tuned model and resume training of the original model's first layer.

  This procedure is designed to mitigate the risk of early convergence to suboptimal local minima, that can happen in joint training of deep networks. By temporally decoupling the optimization of the feature extractor (i.e., the first layer) from that of the final classification layer, the network is encouraged to develop a more robust and generalizable internal representation before any adaptation of the classifier occurs. This separation ensures that the classifier does not prematurely overfit to immature features, which can trap the model in metastable configurations where gradient updates are minimal and generalization is poor.

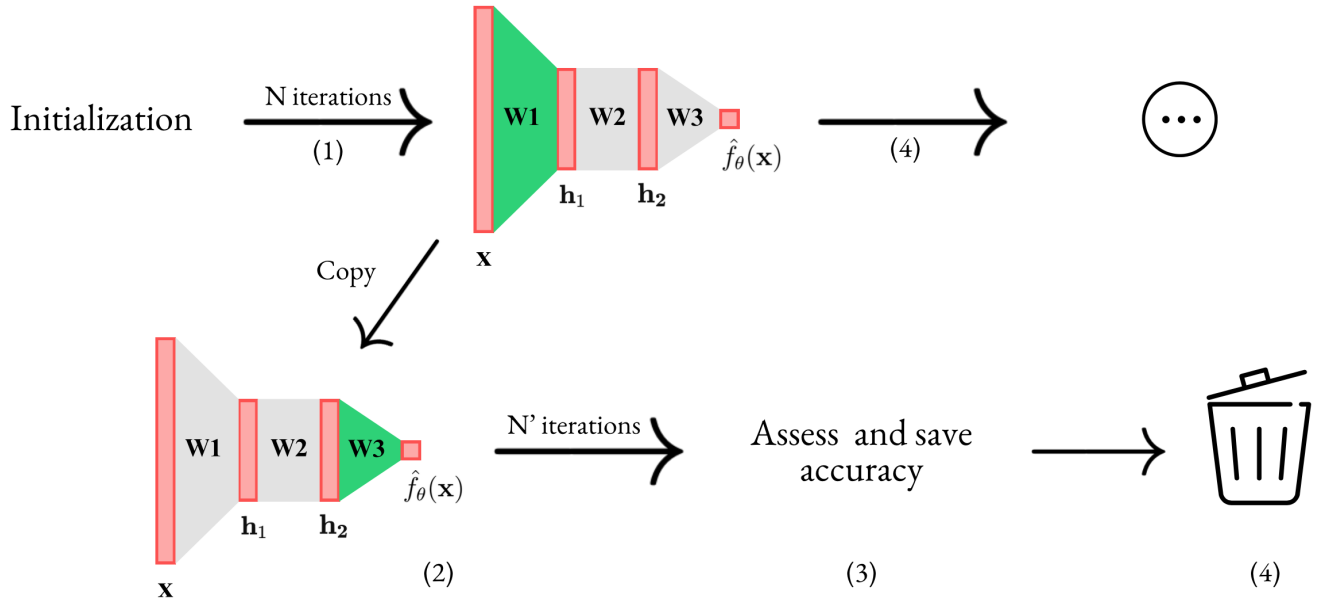
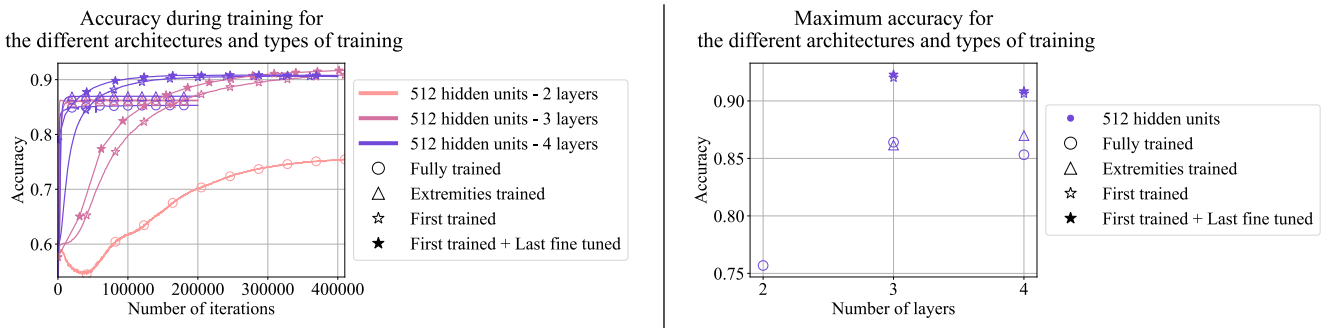


FIGURE 6 – First layer + last layer fine-tuned training strategy.

2.1.3 Results and Analysis :



Accuracy trajectories | Maximum accuracy

FIGURE 7 – Performance comparison on synthetic data (teacher function with $d = 100$, $k = 1$, $n = 10^6$).

Key Observations

1. Fully trained 3-layer networks significantly outperform their 2-layer ones, in line with the expectations of *Dandi et al. (2025)*. This validates the benefit of depth to recover a hierarchical teacher function.
2. Surprisingly, 3-layer models trained only on extremities perform almost as well as fully trained ones, despite having fewer trained parameters. This shows that depth can be leveraged even with partial training.

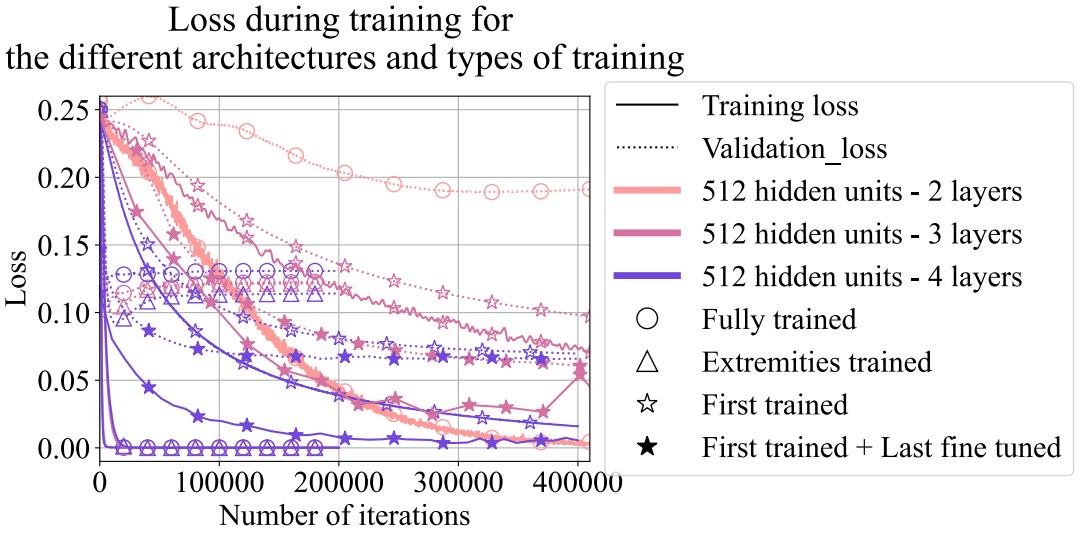


FIGURE 8 – Loss trajectories using MSE loss (training and validation).

3. Even more unexpectedly, models trained only on the first layer eventually surpass the fully trained networks. We hypothesize that these models avoid overfitting and local minima by slowly adapting the feature representation, while fully trained networks converge prematurely to metastable states with poor generalization. Indeed, as shown in Fig. 8, the training loss[1] of fully trained models rapidly drops to zero, indicating overfitting, whereas the training loss of models trained only on the first layer decreases more gradually and remains closer to the validation loss[2] for a longer period.

Conclusions on Synthetic Data.

- Partially trained networks can perform competitively on classification tasks compared to fully trained ones.
- Depth provides benefits through hierarchical learning even when not all layers are trained.
- First-layer-only training may lead to better generalization by avoiding overfitting.

2.2 Real Data (CIFAR-10)

2.2.1 Task Setup :

We now look at real-world data using the CIFAR-10 image dataset, which contains 10 classes. Given the limited capacity of fully connected neural networks (FCNNs) for image

Training Loss[1] : The error computed on the training dataset, which reflects how well the model fits the data it was optimized on.
Validation Loss[2] : The error computed on a separate validation dataset, not seen during training, used to assess the model's generalization ability to unseen data.

data, we restrict our experiments to binary classification tasks.

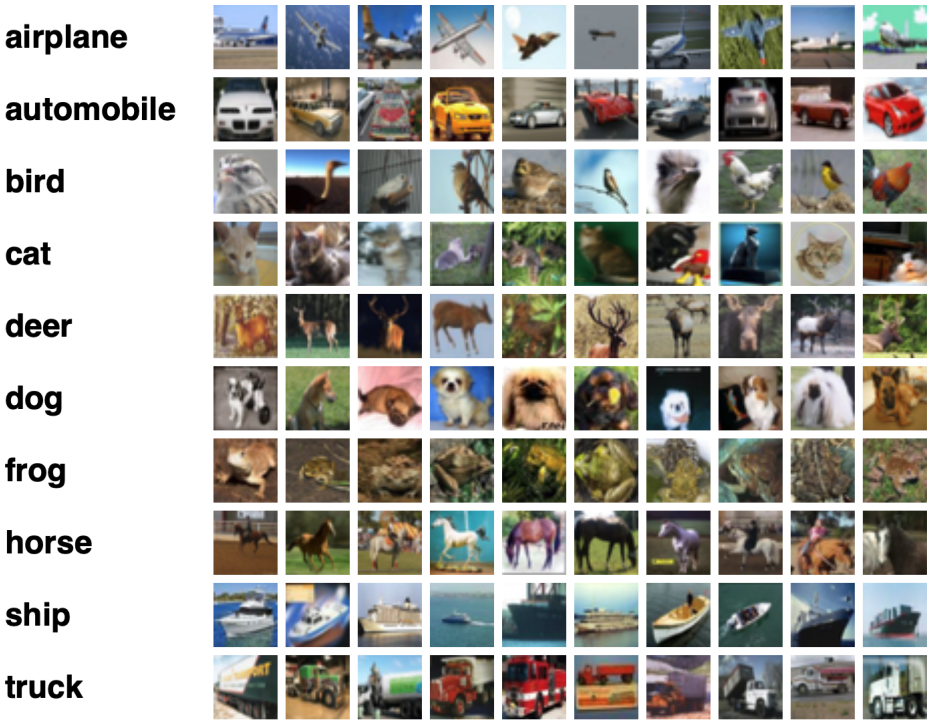


FIGURE 9 – Examples of CIFAR-10 classes.

We test our networks across different widths $W \in \{512, 2048, 8192\}$ and reuse the same training strategies as in the synthetic case.

2.2.2 Results and Analysis :

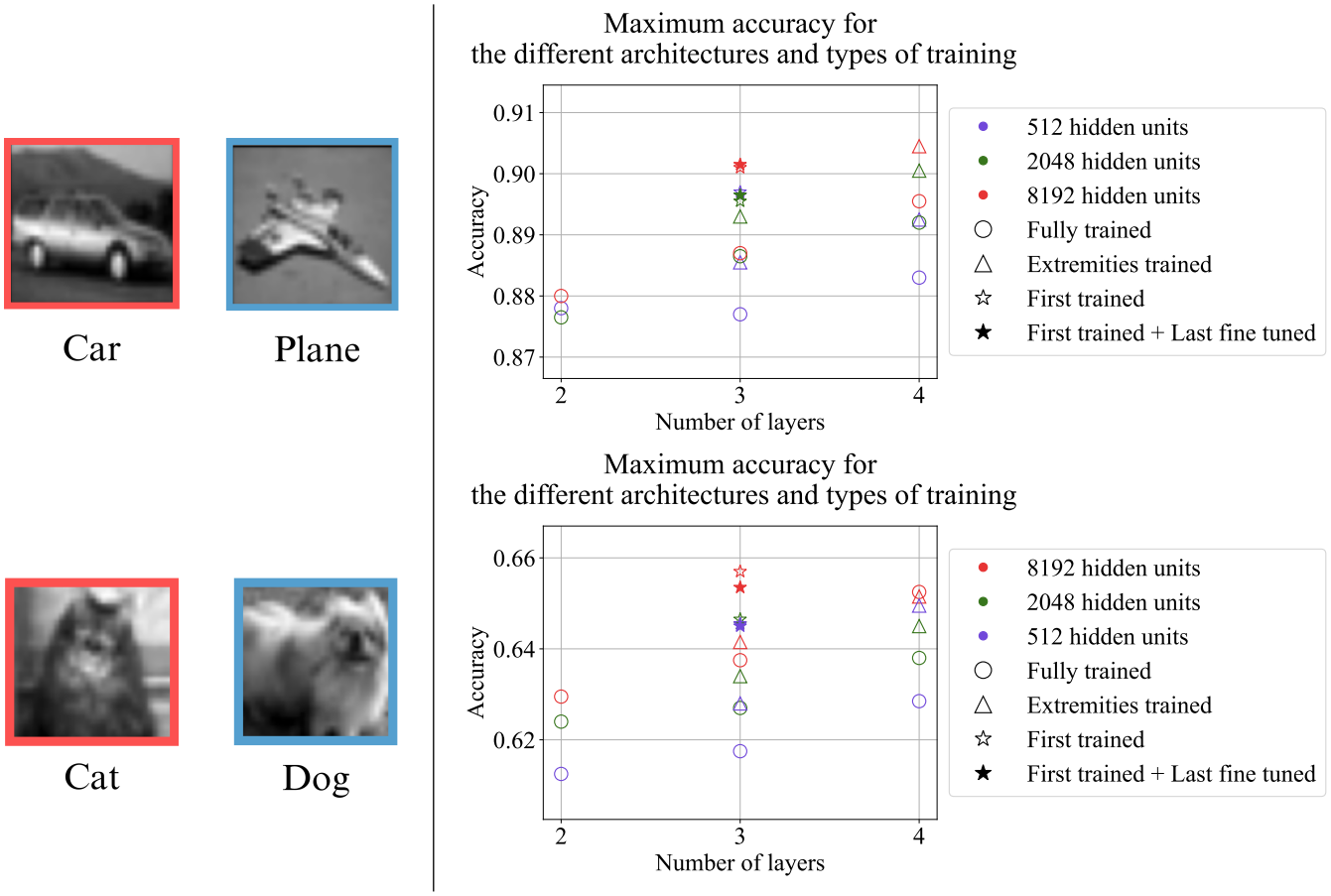


FIGURE 10 – Maximum accuracy achieved on binary CIFAR-10 classification tasks.

Key Observations

1. Fully trained models benefit from increased depth, as one could expect.
2. Extremities-trained models also gain from depth and often outperform fully trained networks.
3. Once again, the first-layer + last-layer fine-tuned strategy gives very performance, consistent with findings on synthetic data.

Conclusions on Real Data

- Partially trained networks are effective on real image data.
- Hierarchical depth remains beneficial even when only a part of layers is updated.

- First-layer-focused training shows superior generalization, possibly due to improved feature extraction.

These experimental results raise a question : can we theoretically demonstrate that training only the first layer of a deep network is sufficient to learn a good representation ? This motivates the next part of the analysis.

3 Theoretical insights for Feature Learning

The objective of this section is to analyze how updating the first layer weights W_1 of a multi-layer perceptron (MLP) influences the feature representation at the second layer, denoted $\mathbf{h}_2$, and, more specifically, how this transformation impacts the correlation between the model's prediction $\hat{f}$ and the target function f^*. This analysis involves the following two steps :

1. Derive the evolution of the second-layer features $\mathbf{h}_2$ induced by a gradient update on the first-layer weights W_1.
2. Determine how this variation in $\mathbf{h}_2$ modifies the correlation between $\hat{f}$ and f^*.

In the remainder of this section, I present the results obtained for the first step of this analysis.

3.1 Model definition :

We consider a three-layer MLP defined by :

$$f(x) = \mathbf{w}_3^\top \, \sigma\big(W_2 \, \sigma(W_1 x)\big)$$

with the following dimensions :

$$x \in \mathbb{R}^d, \quad W_1 \in \mathbb{R}^{d_1 \times d}, \quad W_2 \in \mathbb{R}^{d_2 \times d_1}, \quad \mathbf{w}_3 \in \mathbb{R}^{d_2}.$$

—

Second-layer preactivation We define the **preactivation** of the second layer for neuron i as :

$$h_{2,i}(x) = \mathbf{w}_{2,i}^\top \, \sigma(W_1 x)$$

where $\mathbf{w}_{2,i}$ is the i-th row of matrix W_2.

Feature evolution after a training step If we train the j-th layer of the model slowly enough, we can perform a first-order Taylor expansion of the i-th neuron of the second layer $h_{2,i}$:

$$\Delta_{W_j} h_{2,i}(x) = \left\langle \nabla_{W_j} h_{2,i}(x), \Delta W_j \right\rangle$$

We now attempt to compute the explicit form of $\Delta_{W_j} h_{2,i}(x)$ induced by training of the first layer of the network.

3.2 Evolution of second-layer features after an update of W_1 :

3.2.1 Gradient of preactivation $h_{2,i}$ with respect to W_1 :

We compute :

$$\nabla_{W_1} h_{2,i}(x) = \begin{pmatrix} \vdots \\ \cdots \frac{\partial}{\partial W_1^{k,l}} \sum_m^{d_1} W_2^{i,m} \sigma(W_1 x)_m \cdots \\ \vdots \end{pmatrix}, \quad k \in [d_1],\, l \in [d]$$

$$= \begin{pmatrix} \vdots \\ \cdots \sum_m^{d_1} W_2^{i,m} \sigma'(W_1 x)_m \sum_p^{d} \frac{\partial}{\partial W_1^{k,\ell}} W_1^{m,p} x_p \cdots \\ \vdots \end{pmatrix}$$

$$= \begin{pmatrix} \vdots \\ \cdots W_2^{i,k} \sigma'(W_1 x)_k x_\ell \cdots \\ \vdots \end{pmatrix}$$

In compact form :

$$\nabla_{W_1} h_{2,i}(x) = \mathbf{w}_2^i \odot \sigma'(W_1 x) \odot x \quad \in \mathbb{R}^{d_1 \times d}$$

3.2.2 Expression of ΔW_1 :

We consider the weight update :

$$\Delta W_1 = -\frac{\eta}{n} \nabla_{W_1} \sum_{q=1}^{n} \ell(\mathbf{x}_q)$$

Assuming the output weights are initialized small, we approximate the gradient of the MSE loss as :

$$\Delta W_1 = -\eta \nabla_\theta \mathcal{L}(\hat{f}_\theta, \mathcal{D}) \approx \frac{\eta}{n} \sum_{q=1}^{n} f^*(\mathbf{x}_q) \cdot \nabla_{W_1} \hat{f}(\mathbf{x}_q) = \frac{\eta}{n} \nabla_{W_1} \hat{f}(X) f^*(X)$$

where $X \in \mathbb{R}^{d \times n}$ is the matrix of training samples.

—

3.2.3 Gradient of the output with respect to W_1 :

We compute :

$$\nabla_{W_1} \hat{f}(X) = \nabla_{W_1} \left(\mathbf{w}_3^\top \sigma(W_2 \sigma(W_1 X)) \right) = \sum_{j=1}^{d_2} w_{3,j} \, \nabla_{W_1} \sigma \left(\mathbf{w}_{2,j}^\top \sigma(W_1 X) \right)$$

Unfolding dimensions :

$$\nabla_{W_1} \hat{f}(X) = \begin{pmatrix} \vdots \\ \cdots \sum_j^{d_2} w_{3,j} \, \frac{\partial \sigma(h_{2,j}(X))}{\partial W_1^{k,l}} \cdots \\ \vdots \end{pmatrix}, \quad k \in [d_1],\, l \in [d]$$

$$= \begin{pmatrix} \vdots \\ \cdots \sum_j^{d_2} w_{3,j} \, \sigma' \left(h_{2,j}(X) \right) \frac{\partial h_{2,j}(X)}{\partial W_1^{k,l}} \cdots \\ \vdots \end{pmatrix}$$

$$= \left(\mathbf{w}_3 \odot \sigma' \left(\mathbf{h}_2(X) \right) \right)^\top \left(\nabla_{W_1} \mathbf{h}_2(X) \right)$$

So :

$$\nabla_{W_1} \hat{f}(X) = \left(\mathbf{w}_3 \odot \sigma' \left(\mathbf{h}_2(X) \right) \right)^\top \left(W_2 \odot \sigma'(W_1 X) \odot X \right) \quad \in \mathbb{R}^{d_1 \times d \times n}$$

3.2.4 Increment of preactivations $h_{2,i}$ during training of W_1 :

$$\Delta_{W_1} h_{2,i}(x) = \langle \nabla_{W_1} h_{2,i}(x), \Delta W_1 \rangle = \left\langle \nabla_{W_1} h_{2,i}(x), \frac{\eta}{n} \nabla_{W_1} \hat{f}(X) \cdot f^*(X) \right\rangle$$

$$= \frac{\eta}{n} \sum_{q=1}^{n} \left\langle \nabla_{W_1} h_{2,i}(x), \nabla_{W_1} \hat{f}(\mathbf{x}_q) \right\rangle f^*(\mathbf{x}_q)$$

$$= \frac{\eta}{n} \sum_{q=1}^{n} f^*(\mathbf{x}_q) \sum_{j,k,\ell}^{d_2,d_1,d} W_2^{i,k} \sigma'(W_1 x)_k x_\ell \, w_{3,j} \sigma'(h_{2,j}(\mathbf{x}_q)) W_2^{j,k} \sigma'(W_1 \mathbf{x}_q)_k \mathbf{x}_{\ell,q}$$

$$= \frac{\eta}{n} \sum_{q=1}^{n} f^*(\mathbf{x}_q) \sum_{j=1}^{d_2} w_{3,j} \sigma'(h_{2,j}(\mathbf{x}_q)) \sum_{k=1}^{d_1} W_2^{j,k} \sigma'(W_1 x)_k W_2^{i,k} \sigma'(W_1 \mathbf{x}_q)_k \sum_{\ell=1}^{d} x_\ell \mathbf{x}_{\ell,q}$$

$$= \frac{\eta}{n} \sum_{q=1}^{n} f^*(\mathbf{x}_q) (x^\top \mathbf{x}_q) \sum_{j=1}^{d_2} w_{3,j} \sigma'(h_{2,j}(\mathbf{x}_q)) \sum_{k=1}^{d_1} W_2^{j,k} \sigma'(W_1 x)_k W_2^{i,k} \sigma'(W_1 \mathbf{x}_q)_k$$

$$= \frac{\eta}{n} \sum_{q=1}^{n} f^*(\mathbf{x}_q) (x^\top \mathbf{x}_q) \sum_{j=1}^{d_2} w_{3,j} \sigma'(h_{2,j}(\mathbf{x}_q)) \left(W_2^j \odot \sigma'(W_1 \mathbf{x}_q) \right)^\top \left(W_2^i \odot \sigma'(W_1 x) \right)$$

$$= \frac{\eta}{n} \sum_{q=1}^{n} f^*(\mathbf{x}_q) (x^\top \mathbf{x}_q) \left[\mathbf{w}_3 \odot \sigma'(\mathbf{h}_2(\mathbf{x}_q)) \right]^\top \left[(W_2 \odot \sigma'(W_1 \mathbf{x}_q)) \cdot \left(W_2^i \odot \sigma'(W_1 x) \right) \right]$$

$$= \frac{\eta}{n} \sum_{q=1}^{n} f^*(\mathbf{x}_q) (x^\top \mathbf{x}_q) \left[\nabla_{\mathbf{h}_2(\mathbf{x}_q)} \hat{f}(\mathbf{x}_q) \right]^\top \left[\nabla_{\mathbf{h}_1(\mathbf{x}_q)} \mathbf{h}_2(\mathbf{x}_q) \cdot \nabla_{\mathbf{h}_1(x)} h_{2,i}(x) \right]$$

$$\Delta_{W_1} h_{2,i}(x) = -\frac{\eta}{n} \sum_{q=1}^{n} (x^\top \mathbf{x}_q) \nabla_{\hat{f}(\mathbf{x}_q)} \ell(\mathbf{x}_q) \left[\nabla_{\mathbf{h}_2(\mathbf{x}_q)} \hat{f}(\mathbf{x}_q) \right]^\top \left[\nabla_{\mathbf{h}_1(\mathbf{x}_q)} \mathbf{h}_2(\mathbf{x}_q) \cdot \nabla_{\mathbf{h}_1(x)} h_{2,i}(x) \right]$$

Hence :

$$\boxed{\Delta_{W_1}\mathbf{h}_2(x) = -\frac{\eta}{n}\sum_{q=1}^{n}(x^\top \mathbf{x}_q)\nabla_{\hat{f}(\mathbf{x}_q)}\ell(\mathbf{x}_q)\left[\nabla_{\mathbf{h}_2(\mathbf{x}_q)}\hat{f}(\mathbf{x}_q)\right]^\top\left[\nabla_{\mathbf{h}_1(\mathbf{x}_q)}\mathbf{h}_2(\mathbf{x}_q)\cdot\nabla_{\mathbf{h}_1(x)}\mathbf{h}_2(x)^\top\right]}$$

3.3 Interpretation and Questions :

The expression for $\Delta_{W_1}h_{2,i}(x)$ indicates that the update of $\mathbf{h}_2(x)$ depends on a combination of three factors :

1. the similarity between x and training samples $\mathbf{x}_q$ via the term $x^\top\mathbf{x}_q$.
2. the alignment between the direction of feature change $\nabla_{\mathbf{h}_1(x)}h_{2,i}(x)$ and the direction in which the loss decreases on each $\mathbf{x}_q$.
3. the way the first-layer weights W_1 evolve to reduce the loss via the $\nabla_{\mathbf{h}_1(x)}$.

In particular, the term $\nabla_{\mathbf{h}_1(\mathbf{x}_q)}\mathbf{h}_2(\mathbf{x}_q)$ captures how sensitive $\mathbf{h}_2$ is to changes in the representation $\mathbf{h}_1$, which depends on the evolution of W_1. Therefore, $\mathbf{h}_2(x)$ evolves more in directions where modifying $\mathbf{h}_1$ can simultaneously help reduce the loss on the first and second layers. This implies that the update of $\mathbf{h}_2$ is not random : over training, the evolution of $\mathbf{h}_2(x)$ is fostered toward a value that minimizes the loss over neighborhoods of similar examples, modulated by the other constraints on the learning of W_1. However some questions remains.

Questions : Does this interpretation appropriately capture the role of W_1 in shaping the evolution of $\mathbf{h}_2(x)$? Can this expression be understood as evidence that $\Delta_{W_1}\mathbf{h}_2(x)$ evolves in a non-random direction ? Are other insights to extract from this expression ?

Next steps :

1. See the evolution of the correlation between $\hat{f}$ and f^* when making a $\Delta_{W_1}\mathbf{h}_2$ step.
2. If this doesn't work, try to find a way to show that a $\Delta_{W_1}\mathbf{h}_2$ step contributes to the learning of the task by the MLP.

4 New results with Maximum Update Parametrization (μP)

The previous results already demonstrated that partially trained networks—where only the first layer, or both the first and last layers are updated—can achieve performance comparable

to, or even better than, fully trained deep FCNNs, both on synthetic and real-world datasets. However, after a discussion with a colleague at the laboratory, I discovered a technique that was previously unknown to me, which enables consistent comparison of the performance of networks with different widths. This led to a revised interpretation of some of my earlier findings.

4.1 The need for consistent comparisons across network widths

To compare training dynamics across networks, I initially used a fixed learning rate η regardless of the network width. The update rule for gradient descent is given by :

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta \mathcal{L}(f_\theta, \mathcal{D})$$

The learning rate acts as a scaling factor for the update step. Conventionally, it is chosen to trade-off convergence speed and stability : too small a value leads to slow training, while too large a value may cause divergence or noisy oscillations around local minima. Under this standard view, keeping η fixed across different models seemed like a fair and unbiased choice.

However, empirical results reveal a flaw in this assumption. As shown in Fig.11, the performance of models after a fixed number of training steps varies drastically with both depth and width for a given learning rate. Some architectures perform optimally within certain learning rate ranges, while others become unstable or learn worse.

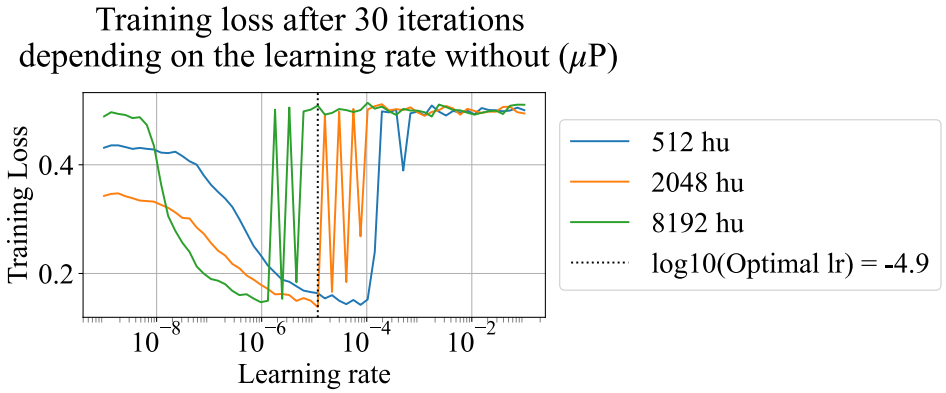


FIGURE 11 – Performance sensitivity to learning rate across different model widths (without μP).

This suggests that selecting a fixed learning rate η that ensures training stability across all ar-

chitectures introduces a bias in the evaluation of model performance. For instance, in Fig. 11, choosing $\eta = 10^{-6}$ results in stable training for all networks. However, under this setting, the learning performance of narrower networks (e.g., those with 512 hidden units) is underestimated compared to that of wider networks (e.g., 8192 hidden units), for which this learning rate happens to be close to optimal.

4.1.1 Maximum Update Parametrization (μP)

Originally developed for large-scale models where tuning hyperparameters is computationally expensive, the μP framework provides a way to ensure consistency across model widths. Its key idea is that, by rescaling the model's parameters appropriately at initialization and during training, one can ensure that models of different widths —but identical depth and structure— exhibit similar training dynamics under a common set of hyperparameters, especially the learning rate.

This allows one to determine the optimal learning rate using a small model and transfer it directly to much larger models with confidence. Fig. 12 illustrates this effect : when using μP, models with varying widths align around the same optimal learning rate.

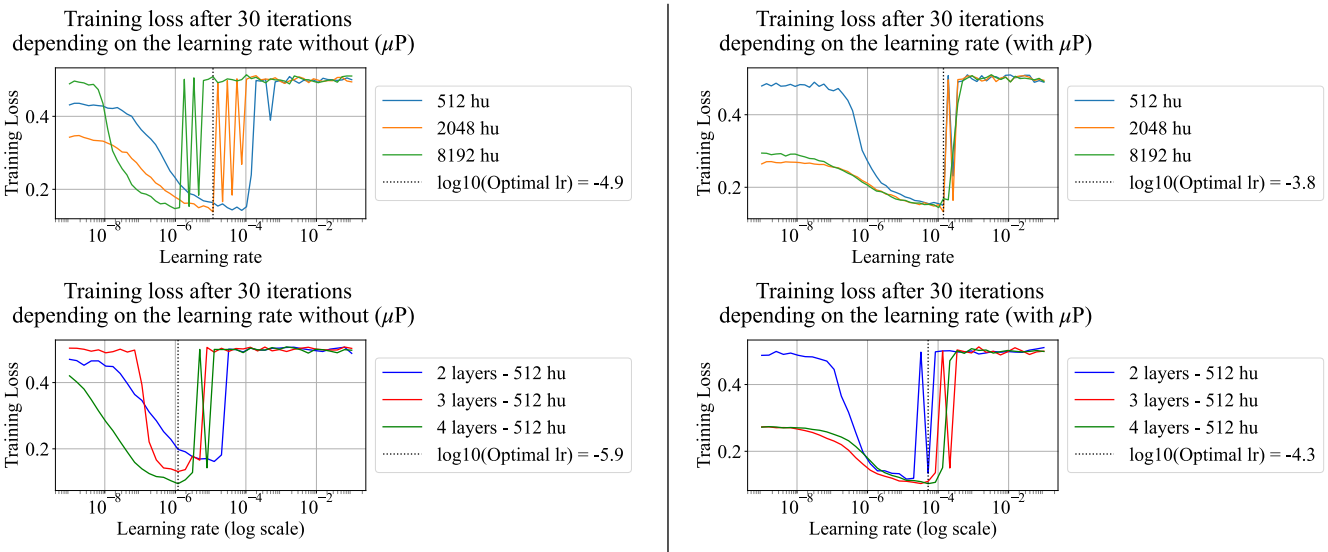


FIGURE 12 – Comparison of model performance on a CIFAR-10 binary classification task with and without μP depending on the width (top) and the depth (bottom) of the network.

4.1.2 Experimental Results with μP

Leveraging μP, I repeated my simulations on real image datasets. The process followed these steps :

1. For each task, determine the optimal learning rate for various model widths under μP.
2. Identify a single learning rate that performs well across different depths at a fixed width.
3. Use this learning rate to rerun the experiments on binary classification tasks.

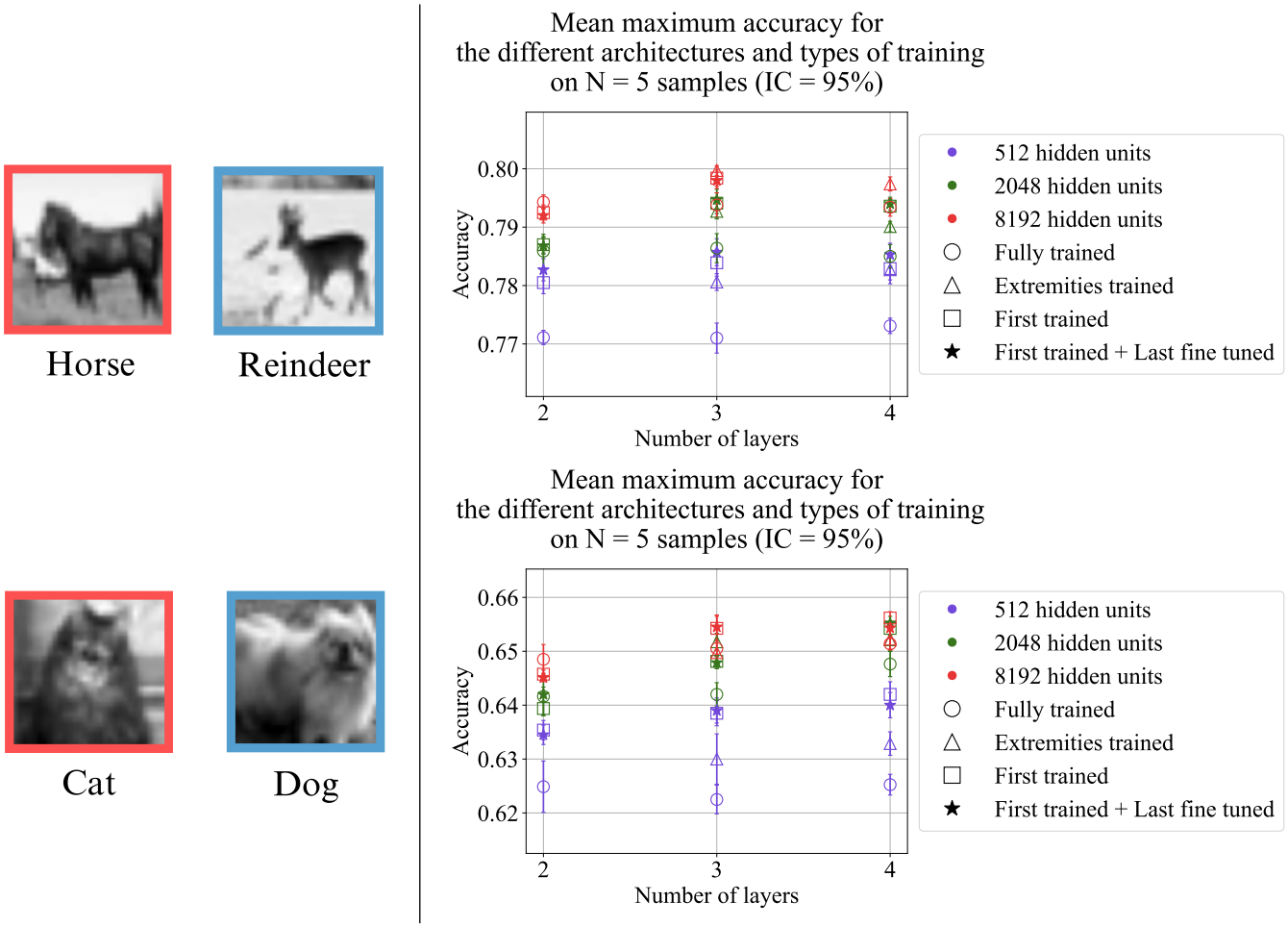


FIGURE 13 – Maximum accuracy as a function of architecture and training regime on binary CIFAR-10 classification tasks, using μP. Each result corresponds to an average over 5 runs.

Observations The results under μP differ significantly from earlier experiments :

- When using an optimal learning rate common to all architectures, fully trained models no longer exhibit any clear advantage from depth. In contrast to previous observations, deeper models no longer consistently outperform shallower ones.

- However, partially trained networks —especially those trained only on the first layer— still achieve competitive accuracy, despite having far fewer trainable parameters. This reinforces the idea that effective feature representations can be learned even with limited parameter updates, and that hierarchical depth remains exploitable even in constrained settings.

Conclusion The introduction of μP has proven crucial for comparisons between networks of varying widths. By ensuring that all models operate under a consistent and effective learning rate, μP eliminates an important source of bias that previously confounded the interpretation of performance differences. While the advantage of increased depth largely disappears under this more rigorous setting when all layers are trained, it is intersting to see that partially trained models —especially those updating only the first layer— continue to yield competitive results. This reinforces the idea that a well-optimized first-layer representation, when combined with sufficient architectural depth, can capture rich hierarchical features with minimal training cost. Consequently, μP not only enhances the reproducibility of experimental findings but also reinforce the evidence supporting partial training as an interesting learning strategy.

Conclusion and Perspectives

In this report, we explored a surprising phenomenon : fully connected neural networks can achieve competitive —sometimes even superior— performance when only their first layer is trained. Through a series of experiments on both synthetic teacher-student datasets and real-world image classification tasks, we demonstrated that this partial training regime is not only viable but can also lead to the emergence of meaningful internal feature representations.

We demonstrated that training only the first layer allows the network to develop meaningful internal representations, even when the remaining layers are fixed and randomly initialized. Moreover, combining first-layer training with last-layer fine-tuning often outperformed fully trained baselines. Introducing muP, those observations were still made in narrower extent, but with a more robust and bias-free analysis. Interestingly, while the benefits of depth disappeared for fully trained models under μP, partially trained networks continued to exhibit strong generalization.

Perspectives This work opens several ways for future investigation. One question is how far this phenomenon extends : up to what depth can a network benefit from training only its first layer ? A deeper theoretical understanding of the dynamics uncovered here is also worth pursuing. More broadly, the effectiveness of partial training raises the question of whether these strategies can be generalized to more advanced architectures such as convolutional neural networks (CNNs) or architectures containing transformers.

These architectures dominate modern applications in computer vision and natural language processing, but they are also notoriously expensive to train. If partial training methods prove to be effective in these settings, they could lead to substantial gains in training efficiency —reducing energy consumption, computational cost, and development time— while maintaining strong performance. Exploring this possibility could lead to very interesting findings for both theoretical and applied machine learning research.

Personal Reflections Beyond the scientific contributions, this internship has been a great personal experience. I had the opportunity to immerse myself in the everyday life of a research laboratory, working alongside PhD students and postdoctoral researchers from diverse backgrounds —France, Italy, Switzerland, India. These exchanges really gave me insights into academic life : the high degree of autonomy required to manage one's research, the flexibi-

lity in organizing workloads, the intense effort before submission deadlines, and the major role of conferences in both sharing work and building academic networks.

Following a previous internship on a more experimental project at the Gulliver Lab, this experience in a highly theoretical setting allowed me to explore the other end of the research spectrum. Even though I worked on one of the most hands-on topics in the lab, I benefited from discussions with colleagues and read many papers related to theoretical machine learning. This helped me clarify what theoretical ML truly is.

While I found the subject very interesting, this experience also raised a growing interest in more applied domains of machine learning—areas driven by short-term objectives and tangible real-world impact. I am now eager to complement my theoretical foundation with hands-on experience in applied settings.

I would like to express my sincere thanks to all members of the lab for their kindness and stimulating conversations throughout this internship. A special thank you goes to Florent for his guidance and mentorship. I will carry excellent memories from this experience—both professionally and personally.